ID3802 – Monthly Progress Report (April)

1 Work done in the current month

1.1 Analysis : Lemma to bound OPT

Here, we try to show that the OPT has to do at least ALG - 2 queries to ensure that the matching produced by it will remain rank-maximal irrespective of how the unrevealed preferences are filled.

1.1.1 Lemma

In RASC-based NRM elicitation, the OPT has to perform at least ALG - 2 queries.

$$OPT \geq ALG - 2$$

We try to produce an instance where the OPT ends up asking 3 queries less than the ALG and claims it has got a rank-maximal matching. We do a case-by-case analysis of the above scenario, disprove the claim of OPT and thus prove the lemma by **contradiction**.

Consider an agent a. The following illustrates the preferences of a revealed by the RASC-based ALG.

Agent	$r=1$	$r=2$	$r=3$	$r=4$	$r=5$	$r=6$	$r=7$	$r=8$	$\cdots$	$r=k$
a	o_1	×	×	$\textcircled{o_4}$	×	$\textcircled{o_6}$	o_7	$\textcircled{o_8}$	$\cdots$	o_k

o_i denotes the i^{th} choice of a and $r = i$ means i^{th} rank.
$k <= n$ (total number of agents or objects) and the agent a is Even till k^{th} iteration (since she is queried till k^{th} iteration).

Recall how ALG works. Some of the preferences of a are not queried by the ALG since the posts are already Odd or Unreachable by the time of that iteration and those skips are marked as **X**. Moreover, the ALG stores the next best choice of a from the available posts in memory in this iteration.

W.L.O.G, we say that in the partial preference list of a revealed by OPT, three choices - o_4, o_6 and o_8 are not revealed (marked in circles) and still OPT claims that it has produced a rank-maximal matching having (a, o), ie., agent a is matched to some object o.

We break down this choice of o into different cases and find a better matching in each case. We leverage the fact that we are free to **manipulate the positions which are not revealed by OPT** and can permute as we wish to prove OPT wrong.

We make use of the concept of **critical rank**[1]. Critical rank c of (a, p) is defined as the first iteration where either the agent a or the object o becomes Odd or Unreachable. For every $1 \leq ic$, the edge (a, p) belongs to every rank-maximal matching, in which (a, p) has rank i

The object o can be classified into five categories comparing its rank with the rank of unrevealed(OPT) objects. In each case, **we promote an object o^* unrevealed by OPT in the preference list to a position which ensures Even-ness for the object, so that $rank(a, p)$ is strictly less than its critical rank. This ensures (a, o^*) is matched in every rank-maximal matching.** We place the promoted object on or before the position where it was queried from. This object has to be Even (available) in this region.

1. $rank(a, o) < rank(a, o_4)$: swap the positions of o_4 and o_6. Every rank-maximal matching should contain (a, o_6) and so we discard the matching given by OPT.
2. $o = o_4$: swap the positions of o_4 and o_6. Every rank-maximal matching should contain (a, o_6). The matching given by OPT no more remains rank-maximal.
3. $o = o_6$: swap the positions of o_6 and o_8. We promote o_8 and match it with a. Every rank-maximal matching should contain (a, o_8) and thus we discard the matching given by OPT. **Here, it makes no sense to promote and use o_6 to match**

with a since $o6$ is the one given by OPT. This is the tighter case which forces us to have three unrevealed objects to efficiently manipulate the list.

4. $o = o_8$: swap the positions of o_4 and o_6. Every rank-maximal matching should contain (a, o_6) and thus we discard the matching given by OPT.

5. $rank(a, o) > rank(a, o_8)$: we can promote either o_6 or o_8 to get a better match for a which is obviously not o.

Here, in each case we could prove OPT is wrong. So, OPT cannot afford asking three or more queries less than the ALG and hence the Lemma 1.1.1 is proved.

1.2 Implementation : Backend for NRM simulator

- We implemented an asynchronous function that takes the number of agent-object pairs as input and returns a Necessarily Rank Maximal matching through an interactive process, by continuously querying users for their preference lists.
- A websocket endpoint '/ws/nrm' was developed which utilizes the function mentioned above and maintains a persistent communciation with the user.
- Apart from the implementation of the algorithm, various error handling mechanisms have also been added to spot inconsistencies in user input. They include:
  - Incomplete user input such as a missing object or object rank.
  - Inconsistent preferences such as input of a lower-ranked object after a higher ranked object or failing to provide a first choice in the first algorithm iteration.
  - Input of a preference that exceeds the number of agent-object pairs that exist.
  - Making a jump in the preference list by providing a less preferred object when a better one is available.
- A frontend for the application is currently under development.

2 Feedback from the mentor

Satisfactory.

3 Signature of the mentor

References

[1] Pratik Ghosal and Katarzyna Paluch. Manipulation strategies for the rank maximal matching problem. October 2017.